

Descrição Resumida

O projeto consiste em um sistema integrado de saneamento e automação de faturamento de recebíveis para unidades de academias (principalmente sob a marca Bodytech e Formula). Ele é estruturado em uma aplicação web desenvolvida com o microframework Flask (Python) no backend e uma interface responsiva baseada em HTML5, JavaScript vanilla e Lucide Icons no frontend. A principal inteligência do sistema é um robô de automação de processos robóticos (RPA) construído com a biblioteca Playwright, que navega de forma assíncrona pelas plataformas EVO/W12 para realizar o login multi-tenant, selecionar unidades de forma robusta, filtrar e emitir Notas Fiscais de Serviço (NFS). Além disso, o sistema conta com um utilitário chamado "Corretor de CPF", que permite a importação de planilhas Excel ou CSV contendo CPFs e realiza a auto-detecção da coluna relevante, preenchendo zeros à esquerda para normalizar os CPFs com 11 dígitos, oferecendo visualização prévia e exportação de arquivos corrigidos.

Problema/Oportunidade

O faturamento de recebíveis e a emissão de notas fiscais de serviço em redes de academias com múltiplos estabelecimentos são processos altamente manuais, repetitivos e propensos a falhas operacionais. A equipe administrativa precisa acessar o portal de faturamento individualmente para cada filial, aplicar filtros de data e regras tributárias complexas, e conferir manualmente se os cadastros dos clientes estão consistentes antes da emissão. Erros comuns como endereços com números ausentes ou preenchidos com letras, bem como a ausência de responsáveis legais configurados para menores de idade sem CPF próprio, travam a emissão de notas fiscais na plataforma. Isso gera atrasos na conciliação, retrabalho administrativo e riscos de inconformidade fiscal. A oportunidade consiste em automatizar e unificar o fluxo ponta a ponta, permitindo a execução autônoma do faturamento em segundo plano ou sob demanda, com inteligência para validar dados cadastrais e corrigir inconsistências em tempo real, reduzindo drasticamente o tempo gasto e aumentando a confiabilidade do processo.

Objetivos

Os objetivos do sistema compreendem tanto aspectos funcionais quanto de arquitetura técnica. Funcionalmente, o sistema visa eliminar a necessidade de execução manual do faturamento diário através de um processo automatizado disparado via agendamento cron ou por acionamento web simples. O robô deve ser capaz de processar sequencialmente as unidades das marcas Bodytech e Formula, identificando cadastros com erros de endereço ou de CPF, aplicando correções automáticas como preenchimento de campos nulos com valores padrão, limpeza de caracteres não numéricos e configuração de responsáveis para menores. O sistema deve emitir as notas válidas, gerar relatórios de auditoria local em formato JSON e enviar um resumo consolidado por e-mail para os gestores, detalhando os casos que não puderam ser corrigidos de forma automática. Tecnicamente, o objetivo é fornecer uma aplicação robusta, com persistência flexível, que execute o robô em segundo plano através de threads dedicadas para não travar a experiência do usuário na

interface web.

Alinhamento Estratégico

A escolha da stack tecnológica baseada em Python, Flask e Playwright está diretamente alinhada à necessidade de portabilidade e baixo custo operacional. O uso do Flask fornece uma arquitetura leve, facilmente containerizável em ambientes Docker, viabilizando o deploy simplificado em servidores locais ou em nuvem. A modelagem de banco de dados por meio do SQLAlchemy assegura a independência do motor de banco de dados, utilizando o PostgreSQL para armazenar de forma segura usuários e logs em produção, ao mesmo tempo em que mantém a compatibilidade com o SQLite para facilitar testes e desenvolvimento local. A automação com Playwright permite a execução headless em servidores Linux ou Windows sem dependência de interfaces gráficas pesadas. A integração estratégica com a API do Gmail garante a notificação passiva dos envolvidos sem necessidade de painéis de acompanhamento adicionais, e a conexão com o RPA Monitor Client centraliza a telemetria, logs de erro e capturas de tela em tempo real, garantindo rastreabilidade técnica essencial para a manutenção contínua e escalabilidade operacional do sistema.

Escopo do Projeto

O escopo atual implementado no código-fonte engloba os seguintes módulos e funcionalidades. No backend, há uma aplicação Flask estruturada com controle de autenticação de usuários e persistência em banco de dados usando SQLAlchemy para as tabelas de usuários (com senha criptografada via Werkzeug hash) e logs de upload. O módulo de automação RPA (Playwright) implementa a lógica de login na plataforma EVO/W12, seleção dinâmica de unidades através de buscas ou varreduras com scroll, navegação para a área de Notas Fiscais de Serviço, aplicação de filtros de data e seleção de múltiplos códigos de tributação (descartando termos inválidos ou sinalizados para não uso). Há um subsistema de higienização de dados que valida cadastros de clientes em solo nacional, corrige números de endereços vazios ou com caracteres alfabéticos (substituindo por zero ou filtrando números), e configura responsáveis legais vinculando checkboxes adequados para clientes identificados como menores de idade sem CPF. O sistema também possui um fluxo para envio efetivo de notas fiscais através da confirmação da data correspondente ao período filtrado e fechamento automático de modais. Há integração com a Gmail API para envio de e-mails detalhados de sucesso ou com a lista de pendências não sanáveis. Por fim, a interface web oferece um dashboard interativo com controle de abas para o acionamento assíncrono do RPA e a ferramenta de correção de CPFs que permite fazer o upload de arquivos Excel e CSV, processá-los na memória com a biblioteca Pandas, exibir um preview formatado na tela com filtros de busca e disponibilizar o arquivo atualizado para download.

Fora do Escopo

O escopo do sistema atual não contempla a geração própria de notas fiscais ou a integração direta via API com a Receita Federal ou com os sistemas das prefeituras municipais para emissão externa, dependendo exclusivamente do portal da plataforma EVO para a efetivação das notas. Também estão fora do escopo funcionalidades de faturamento recorrente automático baseado em cartões de crédito ou cobranças bancárias diretas dentro do próprio Flask, uma vez que o processamento financeiro em si é de responsabilidade da plataforma EVO. Não há no sistema um módulo administrativo completo para o cadastro e edição de novos tenants ou novas regras complexas de faturamento diretamente pela interface gráfica, exigindo alterações no código-fonte para adição de novas academias ou modificação das expressões regulares de seleção de filiais. O sistema também não fornece relatórios gráficos ou de business intelligence complexos em tempo real sobre os valores faturados, limitando-se ao relatório consolidado de auditoria textual e ao arquivo JSON com os registros.

Premissas

As premissas adotadas no desenvolvimento do sistema assumem que as credenciais do sistema EVO/W12 e os dados de acesso da API do Google/Gmail estão corretamente configurados e disponíveis nas variáveis de ambiente descritas no arquivo .env. Assume-se que o banco de dados PostgreSQL estará acessível em ambientes de homologação e produção, possuindo um mecanismo de fallback automático para o banco SQLite local a fim de evitar a paralisação completa do serviço em caso de problemas de rede. O sistema pressupõe a existência de um agendamento cron diário no servidor Linux configurado para disparar o script run_click.sh às 8h da manhã, o qual chama o run_rpa_direto.py em modo headless para garantir a execução autônoma contínua. Para a automação da interface, estabeleceu-se a premissa de que a ordenação por cadastro deve ser aplicada para manter as inconsistências sempre no topo da tabela, e que o scroll da tabela possui um limite de 400 iterações para evitar que falhas de detecção causem loops infinitos no robô. A validação de CPFs da planilha importada manualmente baseia-se na premissa de que a coluna contendo o identificador pode ser inferida de maneira inteligente pela proximidade textual de seu cabeçalho com padrões conhecidos ou pela proporção de dígitos numéricos válidos encontrados nas primeiras linhas do arquivo.

Restrições

Entre as restrições identificadas na análise técnica da solução, destaca-se a forte dependência da estabilidade da interface do usuário da plataforma EVO/W12, de modo que qualquer alteração de layout, nomes de atributos ou seletores CSS/XPath nos componentes da página web pode quebrar o fluxo de automação do Playwright, exigindo manutenção de código. A execução assíncrona do RPA é controlada por threads em Python, o que impõe restrições em ambientes de execução concorrente devido ao Global Interpreter Lock (GIL) e à ausência de uma fila de mensagens ou de gerenciamento distribuído de tarefas mais robusto (como Celery ou Redis) para lidar com múltiplos disparos simultâneos. O monitoramento por imagens em ambientes Linux Headless possui restrições técnicas que levaram à desativação de capturas de tela do PyAutoGUI para evitar travamentos devido à falta de um display server configurado no ambiente de terminal. Outra

restrição de arquitetura é o limite máximo de tamanho de upload de arquivos de planilha fixado em 16 megabytes na configuração do Flask, além da dependência de credenciais do Gmail API persistidas localmente ou através de chaves estáticas no ambiente, o que pode causar falhas de autenticação recorrentes se os tokens de atualização expirarem ou forem revogados pelo provedor de serviços de e-mail.